

DolphinDos26

Manual

Firmware and MS-Tools for the 1541 Switchless 8x Multi-Floppy-Speeder V3 (512 KB)

with MS-NIBCOPY512: nibble copier, G64 import and export

Further development of DolphinDOS 3.0

Frank Eggen – As of 7 November 2026

Contents

1 Introduction....3
1.1 What is DolphinDos26?....3
1.2 About this manual....4
1.3 Files....4
2 The Multi-Speeder....5
2.1 Overview....5
2.2 Switching ROM banks....5
2.3 Drive memory....5
3 Installing the firmware....7
3.1 Programming the EPROM....7
3.2 C64 kernal....7
3.3 First start....7
3.4 Personalisation....8
4 Basic operation (as DolphinDOS 3.0)....10
5 New functions in DolphinDos26....11
5.1 Reading a disk into RAM and writing it back (CDR/CDW)....11
5.2 The RAM disk (CDM)....12
5.3 Faster reading and writing....12
5.4 Bug fixes....13
5.5 MS-Addon-API for your own programs....13
6 The MS-Tools....14
6.1 General....14
6.2 MS-STATUS....14
6.3 MS-MEMTEST512....14
6.4 MS-FASTCOPY512....15
6.5 MS-NIBCOPY512....19
7 DolphinDos26UL....28
8 DolphinDos26DD2....29
8.1 DolphinDos26DD2 and DolphinDos26UL compared....30
9 Performance data....31
10 Quick reference of the new commands....32
11 Notes and limitations....33
12 Appendix....34
12.1 Allocation of the RAM banks....34
12.2 Origins and acknowledgements....34
12.3 Disclaimer....35
12.4 Support through a donation....35
12.5 Copyright and terms of use....35

1 Introduction

1.1 What is DolphinDos26?

DolphinDos26 (DD26 for short) is the drive firmware for the **1541 Switchless 8x Multi-Floppy-Speeder V3** with 512 KB RAM. It is based on the well-known **DolphinDOS 3.0** by Günther Jilg and Jan Bubela. The number 26 stands for the year 2026.

In operation, DD26 remains fully compatible with DolphinDOS 3.0: parallel transfer, track cache, the @ and X commands, the 40-track format and the C64 DolphinDOS kernal work as usual. The new features are mainly functions that use the 512 KB memory of the Multi-Speeder:

- Reading whole disks (35 or 40 tracks) into the RAM of the drive and writing them back to any number of target disks – without transferring data to the C64 (**CDR/CDW**).
- A **RAM disk**: drive 8 works directly from memory (**CDM1/CDM2**); LOAD and SAVE become about three times faster again.
- Faster read and write routines, faster writing back of the directory track, real error messages instead of generic errors.
- A fixed programming interface (**MS-Addon-API**) for C64 programs.
- The **MS-Tools**: C64 programs with a convenient interface for testing the memory, fast copying and nibble copying – with **MS-NIBCOPY512** also exporting as a G64 file and writing G64 files to disk.

There is also **DolphinDos26UL**: a reduced version without Multi-Speeder, with a parallel cable on VIA1, intended as 1541 firmware for the Commodore 64 Ultimate and for real DolphinDOS 2 hardware. It has its own manual (`DolphinDos26UL_Handbuch.odt`, see chapter “DolphinDos26UL”). **DolphinDos26DD2** is DolphinDos26 for the Multi-Speeder with the parallel cable on VIA1 (see chapter “DolphinDos26DD2”).



Figure 1: Power-on screen of the C64 kernal DolphinDos26 (C64DolphinDos26.bin)

1.2 About this manual

This manual describes the new features of DolphinDos26 and the MS-Tools. The basic operation of DolphinDOS (floppy commands with @, X status, function keys, monitor, reset variants) is explained in detail in the original DolphinDOS 3.0 manual; chapter 4 gives a brief summary.

Notation: input is shown in `Schreibmaschinenschrift`. With the DolphinDOS kernal, the easiest way to send commands to the drive is with the at sign, e.g. `@CDR1`. Without the DolphinDOS kernal, the same is done with `OPEN 1,8,15,"CDR1":CLOSE 1`.

> **Note:** Development and testing took place in the VICE emulator with an emulated Multi-Speeder. The DolphinDos26 and DolphinDos26DD2 firmware, the C64 kernal, also as C64 II ROM, and MS-FASTCOPY512 (also with REU storage) run on real hardware; MS-NIBCOPY512 reads and writes normal disks there with MS-RAM and REU, copy-protected disks have not yet been tested on real hardware (see chapter “Notes and limitations”).

1.3 Files

In the `Builds` folder of the project:

File	Contents	Checksum (MD5)
`1541DolphinDos26.bin`	64 KB: DolphinDos26 alone (bank 8 of the EPROM)	`7d2992aa2935c807a023e9474fd6df6f`
`AM27C040-…+7DolphinDos26_v3.3.bin`	512 KB: complete EPROM image – banks 1–7 unchanged from the original EPROM, bank 8 = DolphinDos26	`ed987acb5e413f410d7e6fdb74c91475`
`C64DolphinDos26.bin`	C64 kernal, 8 KB: DolphinDOS parallel, JiffyDOS and serial in one kernal	`5443192b0a3af26062ab1ae78209c0db`
`C1541_DOLPHINDOS26UL.bin`	32 KB: DolphinDos26UL for the C64 Ultimate (VIA1 cable) – **not** for the Multi-Speeder	`e442f8040e0089f10d08edcc6fce353b`
`C1541_DOLPHINDOS26DD2.bin`	64 KB: DolphinDos26DD2 for bank 8 of the 1541 Switchless Multi-Speeder V3. It runs the Multi-Speeder via VIA1 (SpeedDOS cable) instead of its PIA chip, as DolphinDOS 3 intends	`337b632504ad7ad5131f2e745e3073a8`
`ms-tools.d64`	MS-Tools disk (in the `MS-Tools` folder): MS-STATUS (Build 23), MS-MEMTEST512 (26), MS-FASTCOPY512 (29), MS-NIBCOPY512 (49, BETA), MS-RS232 (3); packed with Exomizer	`4a1df9e788ede18ab91b68b96a35f8c4`

The full name of the EPROM image is `AM27C040-1541-0CBMDOS-1DD2-2SpeedDos40TGCR+3JiffyDos5+4S-JiffyDos1+5RapiDOS+6SpeedDos40T+7DolphinDos26_v3.3.bin`.

2 The Multi-Speeder

2.1 Overview

The Switchless 8x Multi-Floppy-Speeder is fitted in the CPU socket of the 1541, between the board and the 6502 CPU (1541-I: socket UC4, 1541-II: socket U3); the original DOS ROMs are removed. It offers:

- 8 DOS ROM banks of 64 KB each in a 512 KB EPROM (27C040/29F040), switchable by DOS command without switches
- a PIA 6821/W65C21 as an 8-bit parallel port to the C64 (DolphinDOS cable on the user port) and for selecting the RAM banks
- 512 KB SRAM (AS6C4008) in 64 banks of 8 KB each, mapped in at $6000–$7FFF
- an additional ROM window $3000–$3FFF for custom DOS extensions (here the MS-Addon of DD26)
- RAMBoard compatibility for nibbler programs

2.2 Switching ROM banks

A microcontroller watches the data bus and changes the ROM bank as soon as the character sequence `n@RNROM` is transferred; the drive then restarts and the LED flashes n times. DolphinDos26 is in the last bank:

```
@I:8@RNROM                          (mit DolphinDOS-Kernal)
OPEN 1,8,15,"I:8@RNROM":CLOSE 1
LOAD"8@RNROM",8,1
```

Bank	Contents (example configuration)	Command
1	CBM DOS 2.6	`1@RNROM`
2	DolphinDOS 2.0	`2@RNROM`
3	SpeedDOS+ 40T + GCR	`3@RNROM`
4	JiffyDOS 5.0	`4@RNROM`
5	S-JiffyDOS (with track cache)	`5@RNROM`
6	SpeedDOS+ 40T	`6@RNROM`
7	RapiDOS Classic	`7@RNROM`
8	**DolphinDos26**	`8@RNROM`

`@I:0@RNROM` disables bank switching until the next power-off – useful if a program happens to send the character sequence by chance.

2.3 Drive memory

Address	Contents
$0000–$07FF	2 KB RAM of the 1541
$1800–$180F	VIA 1 (serial bus)

Address	Contents
$1C00–$1C0F	VIA 2 (drive control)
$3000–$3FFF	MS-Addon of DolphinDos26 (additional ROM), API from $3FFF downwards
$5000–$5003	PIA: port A = parallel port, $5002 = RAM bank selection
$6000–$7FFF	8 KB window into the 512 KB RAM (bank 0–63)
$A000–$BFFF	DolphinDOS (track cache, parallel transfer, X commands)
$C000–$FFFF	CBM DOS 2.6 (901229-05) with DolphinDOS changes

RAM banks: DD26 counts logical banks 0–63. Logical bank 0 is the working RAM of DolphinDOS (variables and track cache) and is active after power-on. Banks 1–50 are occupied by the CDR/CDW functions and the RAM disk (see 5.1). Technically, the bank number is written to the data direction register DDRB of the PIA; because of the pull-up resistors, logical bank n corresponds to physical bank 63−n.

3 Installing the firmware

3.1 Programming the EPROM

The 512 KB image (`AM27C040-…+7DolphinDos26_v3.3.bin`) is written to an AM27C040 (or 29F040) with an EPROM programmer and fitted in the Multi-Speeder. Anyone who only wants to replace bank 8 uses `1541DolphinDos26.bin` (64 KB) in their own image; for the parallel cable on VIA1, use `C1541_DOLPHINDOS26DD2.bin` instead (chapter “DolphinDos26DD2”). For those building it themselves: in the source folder `dd25`, `make builds` regenerates all drive files; in the `C64DD26` folder, it regenerates the C64 kernal.

3.2 C64 kernal

A DolphinDOS kernal is needed on the C64 so that parallel transfer, @ commands and function keys are available. `C64DolphinDos26.bin` is recommended (in a kernal EPROM or a kernal switcher): it addresses DolphinDOS drives in parallel, JiffyDOS devices and SD2IEC with JiffyDOS and all other devices serially, and shows the transfer type when loading (`LOADING DD:` or `LOADING JF:`). The original DolphinDOS 2/3 kernal also works, but it aborts parallel LOAD and SAVE in programs with their own interrupt – Eye of the Beholder, for example, then saves only an empty file of 0–1 blocks and does not load saved games. `C64DolphinDos26.bin` checks the STOP key via the kernal routine STOP and saves and loads such saved games completely. The parallel cable plugs into the user port (label facing up).

3.3 First start

After switching to bank 8, the drive reports `73, DOLPHINDOS26 1541` in the error channel. `@X` shows the X status, e.g. `02,R+:F+:V+:P+:,35,08` (before the first disk access, the track count shows `00`).



Figure 2: Error channel with the power-on message of DolphinDos26 and X status with all switches on

A directory with `@$` and a load show whether parallel transfer is working: with the C64DolphinDos26 kernal, `LOADING DD:` appears with the start and end address.



Figure 3: Directory of the MS-Tools disk with @$ and loading over the parallel cable ("LOADING DD:")

3.4 Personalisation

The colours are single bytes in the ROM and can be changed without an assembler: copy the file `C64DolphinDos26.bin` (8 KB, address $E000 = file offset $0000), change the bytes with a hex editor and use the copy as the kernal. The C64 does not check a kernal checksum. The factory values are those of the original C64 ROM.

Colour	ROM address	File offset	Factory value
Border	$ECD9	$0CD9	$0E (light blue)
Background	$ECDA	$0CDA	$06 (blue)
Text (also start-up screen, `READY.`, cursor, CTRL-V)	$E535	$0535	$0E (light blue)

Colour values for these three bytes:

Value	Colour	Value	Colour
$00	black	$08	orange
$01	white	$09	brown
$02	red	$0A	light red
$03	cyan	$0B	grey 1
$04	purple	$0C	grey 2
$05	green	$0D	light green
$06	blue	$0E	light blue
$07	yellow	$0F	grey 3

All texts of the start-up screen follow the text colour. Only the colour bars have their own colours, as PETSCII control codes in the text:

Bar	ROM address	File offset	Factory code
1	$F001	$1001	$1C (red)
2	$F02D	$102D	$81 (orange)

Bar	ROM address	File offset	Factory code
3	$F03F	$103F	$9E (yellow)
4	$F06B	$106B	$1E (green)
5	$F079	$1079	$9A (light blue)
6	$E473	$0473	$9C (purple)

PETSCII control codes of the colours:

Code	Colour	Code	Colour
$90	black	$81	orange
$05	white	$95	brown
$1C	red	$96	light red
$9F	cyan	$97	grey 1
$9C	purple	$98	grey 2
$1E	green	$99	light green
$1F	blue	$9A	light blue
$9E	yellow	$9B	grey 3

- Do not choose the same text and background colour, or nothing will be readable.
- The control code $06 (`PRINT CHR$(6);`) is new and selects the text colour from $E535. The start-up screen uses it after each bar.
- The addresses apply to this version of the kernal; with later versions compare the existing byte before changing it.
- A changed file has a different checksum from the published one.

4 Basic operation (as DolphinDOS 3.0)

Everything familiar from DolphinDOS 3.0 applies unchanged. The most important commands:

Input	Effect
`@`	read the error channel
`@befehl`	send a command to the drive, e.g. `@s:datei`
`@$`	show the directory
`@x`	extended status R/F/V/P, track count, device address
`@xr-` / `@xr+`	track cache (RAM) off/on
`@xf-` / `@xf+`	compatible or fast LOAD/SAVE
`@xv-` / `@xv+`	verify after writing off/on
`@xp-` / `@xp+`	parallel transfer off/on
`@xt+` / `@xt-` / `@xt`	fixed 40 or 35 tracks, automatic detection
`@x+` / `@x-`	default settings / largely as CBM DOS
`@xl:datei` / `@xu:datei`	lock / unlock file
`@x9",8`	change device address 8 → 9
`@n:name,id` / `@n:name,id+`	format with 35 / 40 tracks
SHIFT+RUN/STOP, F1–F8	load and run, assigned function keys (see the DD3 manual)

If individual programs cause problems, the sequence of steps helps as before: `LOAD"name",8,2` → `@xf-` → CTRL-X → `@xr-` → `@xp-` → switching to another DOS (e.g. `1@RNROM`).

5 New functions in DolphinDos26

5.1 Reading a disk into RAM and writing it back (CDR/CDW)

The drive reads a complete disk into the 512 KB memory on its own and then writes it to one or more target disks. No data is transferred to the C64. The RAM has room for two disks (“side 1” and “side 2”).

Command	Effect
`@CDR1` / `@CDR2`	read the disk into side 1 or 2; 40 tracks are detected automatically (track 36 is checked)
`@CDR1+` / `@CDR2+`	always read 40 tracks (if a track is missing, a read error follows)
`@CDW1` / `@CDW2`	write side 1 or 2 to the inserted disk – as many tracks as CDR has read (35 or 40)

Example – copying a disk three times:

```
@CDR1        Quelldiskette einlegen, lesen
@CDW1        Zieldiskette 1 einlegen, schreiben
@CDW1        Zieldiskette 2 ...
@CDW1        Zieldiskette 3 ...
```

Side	Track 1–35	Track 36–40
1	Bank 1–22	Bank 45–47
2	Bank 23–44	Bank 48–50

- All 683 (or 768) blocks are copied, including those not allocated in the BAM.
- The target disk must be formatted. It also receives the BAM (block 18/0) of the source; the sector headers keep the ID of the target disk. Anyone who wants a matching ID formats the target disk beforehand with the ID of the source – MS-FASTCOPY512 does this automatically on request.
- If the RAM side has 40 tracks, the target disk must also be formatted with 40 tracks (`@n:name,id+`). CDW checks this before the first write and otherwise aborts, leaving the target disk unchanged: `66,ILLEGAL TRACK OR SECTOR,36,00` (only 35 tracks), `21,READ ERROR,36,00` (track 36 missing, with `@xt+`) or `29,DISK ID MISMATCH,36,00` (track 36 holds remains of an earlier format with a different ID, with `@xt+`).
- During CDR/CDW the track cache is always on; afterwards the user’s own setting (XR) applies again. Verify when writing follows the XV setting – with `@xv-`, CDW becomes faster.
- Errors are reported with the real error code, track and sector (e.g. `23,READ ERROR,17,05`); on success the error channel reports `00, OK,00,00`.
- The LED flashes on every bank change.
- The RAM contents survive a reset of the drive, but are lost at power-off.

5.2 The RAM disk (CDM)

With the RAM disk, drive 8 works directly from a RAM side instead of from the disk. Directory, LOAD, SAVE, SCRATCH, opening files – everything works as usual, only without motor and head.

Command	Effect
`@CDM1` / `@CDM2`	switch drive 8 to the RAM disk (contents of side 1 or 2)
`@CDM0`	back to the disk

Typical procedure:

```
@CDR1          Diskette nach Seite 1 lesen
@CDM1          ab jetzt arbeitet Laufwerk 8 aus dem RAM
LOAD"*",8,1    ... laden, speichern, ändern ...
@CDM0          zurück zur Diskette
@CDW1          geänderten Inhalt auf Diskette sichern
```



Figure 4: CDR1 reads the disk into RAM; after CDM1 the directory comes from the RAM disk

- Fast loading and saving (XQ/XZ) run directly from and to the RAM disk: loading 30 KB takes 1.2 s (disk 3.7 s), saving takes 1.9 s (disk 7.3 s).
- `@n:name,id` in RAM mode clears the RAM side (empty disk in RAM).
- During RAM mode, XR and XF are switched off internally, because the fast paths of DolphinDOS would otherwise read the disk directly; after `@CDM0` the user's own setting applies again.
- RAM mode survives a reset of the drive (the power-on message appears, the directory still comes from RAM). Only `@CDM0` or power-off ends it.

> **Important:** Changes in the RAM disk are lost at power-off – save them first with `@CDW1`/`@CDW2`. MS-MEMTEST512 and MS-NIBCOPY512 (MS-RAM storage) overwrite the RAM.

5.3 Faster reading and writing

DD26 contains several optimisations of the read and write paths. They affect CDR/CDW and in part also normal LOAD/SAVE:

Improvement	Effect
GCR decoding in the track cache unrolled	reading a sector 12.0 → 10.4 ms
write buffer copy optimised	writing a sector 11.8 → 10.5 ms
CDW writes the directory track (18) via the write-back cache	track 18 in 0.9 s instead of 8.7 s
CDW no longer reads the target track first	approx. 0.2 s less per track
job is started immediately (controller IRQ triggered)	up to 15 ms less waiting time per job
switching the RAM bank once per sector instead of once per byte	copying 9.7 → 3.8 ms per sector

5.4 Bug fixes

- **Formatting (N:):** Moving to track 1 (bump) left the stepper motor in a wrong phase, depending on the head position; half steps were then lost and formatting aborted at track 19. DD26 keeps the phase or, if the position is unknown, moves fully to the stop.
- CDR/CDW report real error codes with track and sector instead of a generic `74,DRIVE NOT READY`.
- After a successful CDR/CDW, `00, OK` is in the error channel.
- The overflow of the RAM banks is checked correctly against the last bank.

5.5 MS-Addon-API for your own programs

C64 programs can call the functions of the addon via fixed entry points. The table is at the end of the addon ROM and stays in the same place in future versions; new functions are added below it.

Address	Function	Parameters / result
$3FFF	Version	1
$3FFE	Number of entry points	7
$3FFC	Identifier	"MS"
$3FF9	E0 OPEN	side 1/2, direction 0 = read / 1 = write, verify 0 off / 1 on / 2 as XV. Writing a 40-track side: target checked as with CDW (66, 21, 29)
$3FF6	E1 TRACK	read or write track 1–40 (RAM location as CDR1/CDR2)
$3FF3	E2 CLOSE	track count 35/40; when writing, the last track is written back
$3FF0	E3 PROBE	track: seek job, 00 = header with the ID of the disk found, different ID: 29
$3FED	E4 INFO	message `00, OK,t1,t2` = track count of sides 1 and 2
$3FEA	E5 ABORT	clean up after an error (cache, XR/XV as before OPEN)
$3FE7	E6 SECTOR	copy side, track, sector from RAM to the buffer $0300 (then M-R)

Call: M-E with address and up to three parameters on channel 15, i.e. the bytes "M-E", low byte, high byte, p1, p2, p3. The result is in the error channel. Check the identifier and version first with M-R from $3FFC.

6 The MS-Tools

6.1 General

The MS-Tools are C64 programs (C64 mode; on the C128 after `GO 64`) on the disk `ms-tools.d64` (name MS-TOOLS, ID 26). They are packed with **Exomizer** and unpack themselves after `RUN` (as a result, MS-NIBCOPY512 occupies 58 instead of 102 blocks). Each tool loads like a BASIC program:

```
LOAD"MS-FASTCOPY512",8
RUN
```

`LIST` still shows the name, build number, year, author and device, e.g.:

```
10 REM MS-FASTCOPY512 BUILD 29 / 2026
20 REM BY FRANK EGGEN
30 REM SWITCHLESS MULTI-SPEEDER V3
40 SYS2153
```

Operation (the same for all tools, texts in English):

- red screen, title bar with name and build at the top, a help line at the bottom
- CRSR up/down selects a menu item (yellow bar), RETURN executes it
- CRSR left/right (or RETURN) changes settings such as DRIVE, TRACKS or VERIFY
- RUN/STOP aborts a running operation, ← exits; on exit, the border, background and text colours are set back to what they were before the start
- the default is the drive from which the tool was loaded (DRIVE 8–11)

6.2 MS-STATUS

Shows the error channel message of the selected drive (menu READ STATUS / QUIT). Handy for quickly checking whether the drive is ready, and as a template for your own tools.

6.3 MS-MEMTEST512

Tests the 512 KB SRAM of the Multi-Speeder. The test program runs in the drive; the C64 shows the result for each bank in an 8×8 grid.

- Menu: START TEST, RUNS (1–99 runs), DRIVE (8–11), QUIT
- Four patterns per run: $55, $AA, an address-dependent pattern and its inverse. First all banks 1 –63 are filled, then all are checked – this also reveals overlapping banks or address line faults. Bank 0 (working RAM) is saved at the end and also checked.
- Grid: grey = pending, yellow = being tested, green = OK, black = ERROR (remains across all runs); in addition, the number of faulty bytes and the first error with bank, address, expected and actual value
- One run takes about 2 minutes.



Figure 5: MS-MEMTEST512 after one run: all 64 banks OK

> **Warning:** The test overwrites the entire RAM contents (CDR sides, RAM disk). An active RAM disk is first switched off with CDM0.

6.4 MS-FASTCOPY512

Convenient disk copy via the RAM of the Multi-Speeder (data copy like CDR/CDW, via the MS-Addon-API) or via a REU, with a progress display for each track. The menu has two columns like MS-NIBCOPY512: SETTINGS on the left, ACTIONS on the right. **CRSR up/down** moves through all items, **CRSR left/right** changes a setting or jumps from right to left.

6.4.1 Minimum hardware requirements

Component	Minimum requirement
C64	C64 or C128 in C64 mode
Storage (required)	MS-RAM (via the MS-Addon API) or REU – one of the two is needed; without storage nothing is copied
Drive	1541 with Switchless Multi-Speeder V3 and DolphinDos26 or DolphinDos26DD2 (MS-Addon API) for MS-RAM storage; without the API (e.g. DolphinDos26UL) only with an REU
Connection and kernal	for REU storage only: active parallel cable (PIA on the Multi-Speeder, VIA1 with DolphinDos26DD2 and DolphinDos26UL) and a DolphinDOS kernal on the C64, `C64DolphinDos26.bin` recommended; without a parallel connection the REU is not offered
REU	for REU storage only: one slot (`REU 1`) from 256 KB, two slots from 512 KB
Otherwise	without the MS-Addon API and without a usable REU the program only shows the directory (DIRECTORY); copying is not possible

6.4.2 Operation

Setting	Meaning
STORAGE	memory for reading and writing: `MS-RAM 1`/`MS-RAM 2` (the two RAM sides of the Multi-Speeder) or `REU 1`/`REU 2` (256 KB each; one slot from 256 KB of REU, two from 512 KB). Only what is usable is offered.
TRACKS: 35/40/AUTO	track count; AUTO checks track 36 like CDR
VERIFY: ON/OFF	verify after writing
DRIVE: 8–11	drive; on change, a check is made whether DolphinDos26 with API is running there (otherwise `NO MS-ADDON API`) and whether transfer is parallel (`PARALLEL: PIA` or `PARALLEL: VIA1`, otherwise `NO PARALLEL`)

Action	Procedure
COPY DISK	read the source, then write to the target disk
READ DISK	read only
WRITE DISK	write only
IMPORT D64 FILE	load a **D64 file** from a drive (e.g. SD2IEC) into the selected memory, then write it to disk with WRITE DISK
EXPORT D64 FILE	write the contents of the selected memory as a **D64 file** to a drive
DIRECTORY	show the directory of the disk in the selected drive (CRSR up/down: line, CRSR left/right: page, RETURN: back); also works without the MS-Addon-API



Figure 6: Main menu of MS-FASTCOPY512

- After each write, the tool asks `ANOTHER COPY?` – so one read can produce any number of copies.
- If the target disk is not formatted, the tool asks `NOT FORMATTED - FORMAT 35 TRACKS?` (or 40) and formats with the name and ID of the source disk, so that sector headers and BAM match.
- Grid of tracks 1–40: grey pending, yellow current, green done, black error; below it the activity, the message of the drive and the track count of both slots of the selected memory.
- The red drive LED stays lit during the entire read and write and goes out afterwards.



Figure 7: MS-FASTCOPY512 after READ DISK: disk read into MS-RAM 1

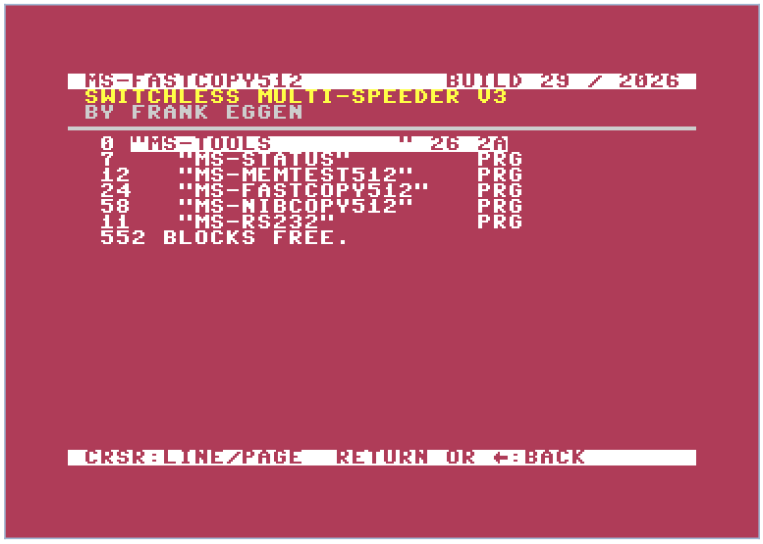


Figure 8: MS-FASTCOPY512: disk directory via DIRECTORY

6.4.3 Speed

Measured on real hardware (C64U with a 1541 with Multi-Speeder V3, kernal `C64DolphinDos26.bin`), 35 tracks, time from RETURN to the end, to the nearest second:

Firmware (cable)	Storage	Reading	Writing with verify	Writing without verify
DolphinDos26 (PIA) [1]	MS-RAM	approx. 24 s	approx. 30 s	approx. 22 s
DolphinDos26 (PIA) [1]	REU	approx. 82 s	approx. 120 s	–
DolphinDos26DD2 (VIA1)	MS-RAM	24 s	27 s	20 s
DolphinDos26DD2 (VIA1)	REU	82 s	120 s	113 s
DolphinDOS 2.0 (VIA1) [2]	REU	82 s	121 s	114 s

- [1] With a different 1541 and a coarser measurement (about ± 6 s); writing without verify with an REU not measured.
- [2] Original DolphinDOS 2.0 from bank 2 of the Multi-Speeder EPROM (`@I:2@RNROM`), REU only, because it has no MS-Addon API.

- With MS-RAM the track stays in the drive; with an REU every sector travels over the parallel cable to the C64 and back. The cable (PIA or VIA1) makes hardly any difference.

6.4.4 REU storage

The REU is an alternative to the Multi-Speeder RAM and also works with drives without a Multi-Speeder, for example DolphinDos26UL on the VIA1 cable. It requires a **parallel connection**: the program checks it at startup and on every drive change over the whole path (C64 kernal, cable, firmware). Without it (`NO PARALLEL`, also with `@XP-` or a kernal without DolphinDOS) the REU is not offered.

- The sectors travel one by one through a direct-access channel (`U1`/`U2`) and the C64. This is slower than MS-RAM: in the emulator with verify about 90 s reading and 130 s writing for 35 tracks, 120 s and 160 s for 40 tracks (MS-RAM: see the chapter “Performance data”).
- The data stays as long as the REU has power, even after restarting the program. A marker in the slot remembers the track count; an empty slot reports `NOTHING READ INTO THIS REU SLOT`.
- MS-NIBCOPY512 uses the same REU from the bottom, MS-FASTCOPY512 from the top. With a small REU the data of the two programs can overwrite each other; this is not always detected.
- When reading with TRACKS: AUTO, the LED can flicker briefly at the end: checking track 36 reports read errors on 35-track disks.

6.4.5 D64 files

The dialogs match those of MS-NIBCOPY512 for G64 files. A D64 file is 175 KB and does not fit on a 1541 disk; the target or source is therefore another device, typically an **SD2IEC**.

1. For export, read the disk first (READ DISK); for import, select the desired memory (STORAGE).
2. Select **EXPORT D64 FILE** or **IMPORT D64 FILE** and press RETURN.
3. `EXPORT TO DRIVE: 09` or `IMPORT FROM DRIVE: 09`: choose drive 8–11 with **CRSR left/right**, **RETURN** accepts (a missing device reports `DRIVE 10: NOT PRESENT`).
4. `FILE NAME:`: type the name (up to 12 characters: A–Z, 0–9, space, `-`, `.`, `+`; **DEL** deletes). The extension `.D64` is appended; on export an existing file is replaced. **RETURN** starts, ← cancels.
5. The grid shows the progress. At the end you see `EXPORT OK: NAME.D64` or `IMPORT OK: 35 TRACKS` and the message of the drive.

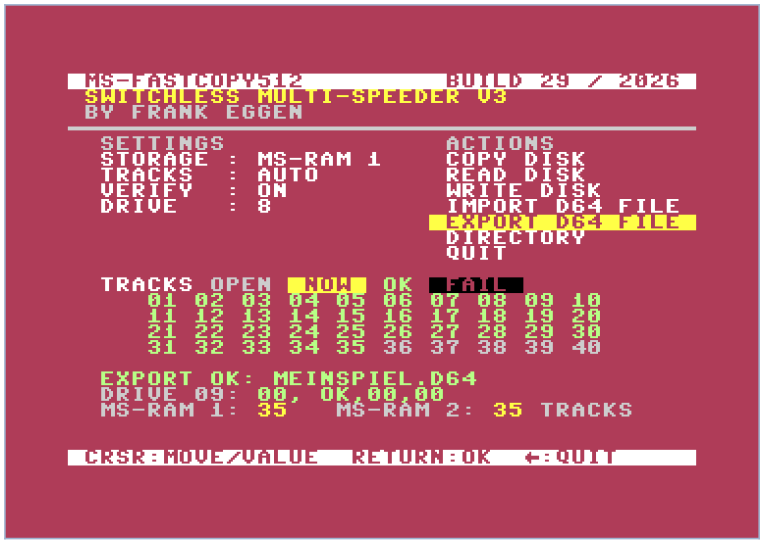


Figure 9: MS-FASTCOPY512: EXPORT D64 FILE finished (drive 9, e.g. an SD2IEC)

- Import accepts D64 files with 35 or 40 tracks, also with an error table (it is ignored). Other sizes report `IMPORT FAILED: NOT A VALID D64 FILE`.
- With MS-RAM storage, the drive holding the RAM cannot also be target or source (`MS-RAM USES THIS DRIVE - CHOOSE ANOTHER`). With REU any drive is allowed.
- If an import into MS-RAM is aborted or the file is invalid, the slot is partly overwritten but still shows the old track count. A REU slot then counts as empty.

> **Note:** This has been tested in the emulator, with a directory as drive 9. A real SD2IEC normally treats `.D64` files as disk images; whether it also writes and reads them as plain files has not been checked yet.

6.5 MS-NIBCOPY512

A nibble copier modelled on **nibtools** (Pete Rittwage), ported to the C64. A small program (payload, 1279 bytes) is loaded into the RAM of the drive and reads the tracks bit-accurately as raw data – including syncs, gaps, non-standard formats, tracks 36–42 and half tracks. The C64 does the analysis (track length, alignment, density, sector check, weak bits). This makes it possible to copy many copy-protected disks too. Disks that have been read can also be saved as a **G64 file**, and G64 files can be written to disk.

> **BETA:** MS-NIBCOPY512 is in BETA and still under development. Functions and operation may still change, and it has only been partly tested on real hardware. The user interface therefore shows `MS-NIBCOPY512 BETA`.

6.5.1 Minimum hardware requirements

Component	Requirement
C64	C64 or C128 in C64 mode
Storage (required)	MS-RAM or REU – one of the two is needed; without storage the program can neither read nor write

Component	Requirement
Drive	Multi-Speeder with DolphinDos26 (the payload detects the PIA itself; the program also runs with DolphinDos26UL on the VIA1 cable, but then only with REU; with DolphinDos26DD2 it uses the VIA1 cable and MS-RAM or REU); not tested with DolphinDOS 2
Connection	parallel cable active (`P+` in the X status); without a cable the program reports `NO ANSWER FROM DRIVE CODE - CABLE?`
Storage	**MS-RAM** (default): the raw data of each track (up to 8 KB) is held in the RAM of the Multi-Speeder, track t in bank t, half tracks in bank 43–62, bank 63 is the read and write buffer. Alternatively **REU** in the C64 (64 KB–16 MB, size is detected).
Program	`MS-NIBCOPY512` on the disk `ms-tools.d64`

6.5.2 Starting and main menu

```
LOAD"MS-NIBCOPY512",8
RUN
```



Figure 10: Main menu of MS-NIBCOPY512 on the Multi-Speeder: settings on the left, actions on the right, storage in MS-RAM

After starting, the program loads the saved presets (`MS-NIBCOPY.CFG`) and shows the main menu in **two columns**: **SETTINGS** on the left, **ACTIONS** on the right. **CRSR up/down** goes through all items in sequence (from STORAGE on to COPY DISK), **CRSR left/right** changes a setting or jumps from the right column to the left, **RETURN** executes, ← leaves the menu or the program, **RUN/STOP** aborts a running operation.

> While MS-NIBCOPY512 is reading or writing, the **red LED** of the drive is lit. Afterwards DolphinDos26 takes over the LED again itself.

Menu item	Meaning
SETTINGS (left)	
PRESET	STANDARD (track 1–35, without half tracks), EXTENDED (up to track 42 with half tracks), CUSTOM 1–3 (freely configurable)

Menu item	Meaning
END TRACK	last track: STANDARD 35–40, EXTENDED 36–42, CUSTOM 1–42. After IMPORT G64 FILE this shows the last track of the file; the value then also applies to the next read (permanently only with SAVE PRESETS)
HALFTRACKS	read half tracks too (empty and duplicate ones are discarded)
EDIT PRESET …	submenu (see below)
DRIVE	drive 8–11; if no device answers, `DRIVE nn: NOT PRESENT` appears
STORAGE	storage: `MS-RAM` (default) or `REU` with detected size, e.g. `REU 512K`
ACTIONS (right)	
COPY DISK	read the source, then write to the target disk
READ DISK	read only (into MS-RAM or REU)
WRITE DISK	write the memory contents to disk (tracks read or imported); then `ANOTHER COPY?` for further copies
IMPORT G64 FILE	load a **G64 file** from a drive (e.g. SD2IEC) into memory, then write it to disk with WRITE DISK
EXPORT G64 FILE	write the tracks read as a **G64 file** to another drive, e.g. an SD2IEC
TRACK INFO	table per track: density, length, alignment, syncs, faulty sectors, read attempts (page by page with CRSR)
DIRECTORY	show the directory of the disk in the selected drive (CRSR up/down: line, CRSR left/right: page, RETURN or ←: back); tracks read or imported into storage are kept
QUIT	exit the program

6.5.3 EDIT PRESET

Setting	Meaning
START TRACK / END TRACK	first and last track (1–42)
HALFTRACKS	half tracks on/off
ALIGNMENT	start of track when writing: AUTO, GAP (largest gap), SECTOR 0, LONGSYNC, BAD GCR, AUTO GAP, RAW – as in nibtools
DENSITY	AUTO (determine the density per track) or fixed 0–3
RETRIES	read attempts 1–9 (default 5)
VERIFY	read back and compare each written track (default ON, up to 3 attempts)
RESET TO DEFAULTS	restore the defaults
SAVE PRESETS TO DISK	save the settings as `MS-NIBCOPY.CFG` to the tool disk (`INSERT MS-TOOLS DISK`)
BACK	back to the main menu

6.5.4 Copying a disk

1. Select PRESET: STANDARD for normal disks, EXTENDED for disks with tracks above 35 or half tracks.
2. Select **COPY DISK** and press RETURN.

3. `INSERT SOURCE DISK`: insert the original disk, RETURN. The tracks are read; the grid shows the progress (OPEN pending, NOW in progress, OK, SPECIAL special track such as a killer track or one without sync, ERR error). At the end, `READ OK: TRACKS 01-35` appears.
4. `INSERT TARGET DISK`: insert the target disk, RETURN. The program first measures the capacity of the target disk and then writes each track, with verify.
5. After `WRITE OK` the program asks `ANOTHER COPY?`: RETURN for another copy, ← to finish.



Figure 11: After READ DISK: track 1–35 read and stored in MS-RAM

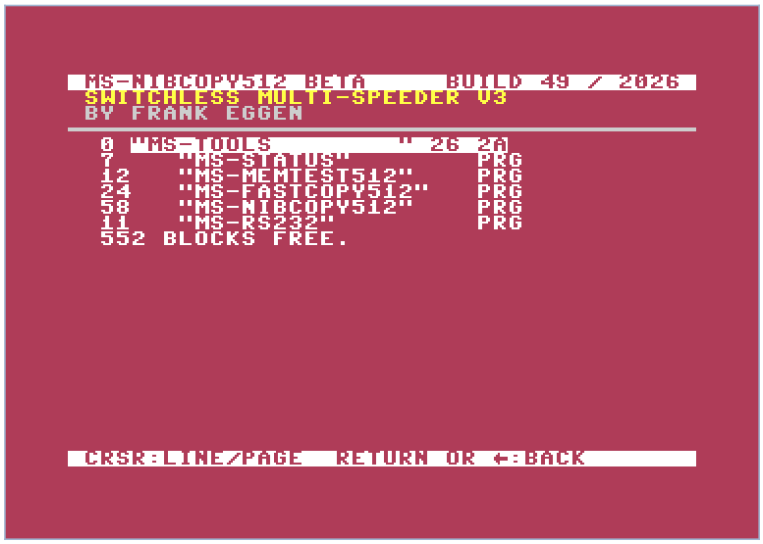


Figure 12: MS-NIBCOPY512: directory shown with DIRECTORY

Message	Meaning
`TARGET DISK IS WRITE PROTECTED`	target disk is write-protected
`DRIVE SPEED NOT 280-320 RPM`	drive rotation speed outside the permitted range
`VERIFY FAILED`	track not written correctly after three attempts
`DRIVE nn: NOT PRESENT`	no device answers at the selected drive
`NO DIRECTORY - SEE DRIVE MESSAGE`	DIRECTORY: no directory could be read, e.g. on an unformatted disk; the drive message is shown below
`NO NIBBLE DATA IN STORAGE`	nothing has been read or imported yet

Message	Meaning
`REU TOO SMALL FOR THE END TRACK`	REU storage only: too small for the selected last track
`NO ANSWER FROM DRIVE CODE - CABLE?`	no connection over the parallel cable

Times (emulator): reading and analysing a track approx. 1.5 s, writing with verify approx. 2.5 s.

6.5.5 Weak bits

Some copy protection methods work with “weak bits”: places on the track without flux transitions (three or more zero bits in a row, i.e. invalid GCR). After about 18 µs without a flux transition, the amplifier gain control of the drive generates its own flux transitions, so the drive reads random bits there. The protection reads such a place several times and checks whether it reads differently each time. An ordinary copy, by contrast, contains the bits as read once, always reads the same and is detected as a copy.

MS-NIBCOPY512 detects and copies weak bits automatically; no setting is needed for this:

- If a track that has been read contains invalid GCR, it is read three additional times. Each of these reads is aligned to the first, anew at each sync.
- Bytes that differ between the reads and already contain invalid GCR in the first read (at most three in a row) are candidates. They count as weak bits only when **three reads** report them; this way, borderline cases that only rarely read differently are not wrongly replaced. If there are still unconfirmed candidates after the three reads, up to seven more follow.
- Weak bits are stored in the raw data as $00, as in nibtools.
- When writing, areas without flux transitions are created again at these places. There, like the original, the copy reads differently on each read.
- Verify skips the weak-bit places, because they cannot be compared.
- Tracks with invalid GCR take longer to read (three to ten additional reads and the comparison on the C64). A disk with many such tracks takes around 20 minutes to read in the emulator.

> In the VICE emulator, weak bits read only weakly at random: a place there often reads the same several times and is then occasionally not detected (with one test image, about every fourth copy does not start). In this case, repeat the copy. On real drives, weak bits presumably read considerably more randomly; this has not yet been tested.

6.5.6 Four and five zero bits

Some protection methods contain places with four or five zero bits in a row. Every 1541 reads them falsified: after three zero bits without a flux transition, its bit counter itself generates a 1, turning “1 0000 1” into “1 0001 1”. At normal density, the original cannot be distinguished from this falsified form; a copy of the bits read is wrong.

The bit counter restarts at each flux transition. At a different density (different clock), the drive therefore reads the same intervals as different but fixed bit sequences. MS-NIBCOPY512 uses this automatically:

- If a track with density 3 contains such places, it is read one additional time with density 0.
- For each place, the C64 forms the bit sequence that each of the two possibilities would produce at density 0 (with the bits before and after), and searches for it bit by bit in this read. If only the

sequence with four or five zero bits is found, the 1 inserted by the drive is deleted; otherwise the place stays as it was read.

- This costs about 2 seconds per affected track. Tracks of other densities are not checked in this way.

Two further changes are connected with this:

- If no cycle is found in a track (track without sync), it is read up to six times instead of twice. In the emulator, a place with five zero bits reads with one extra bit in every second read; the track then does not repeat in whole bytes.
- With alignment on the longest gap (AUTO GAP, and with AUTO for tracks without sectors), for gaps of 48 bytes or more the track starts 16 bytes after the start of the gap, instead of 5 bytes before its end as in nibtools. This keeps the seam of the copy from lying directly in front of what a loader checks behind the gap. Tracks without sync are filled at the seam with the pattern of the gap instead of with $55.

> This has been tested in the VICE emulator (the affected track of a test image is copied bit-accurately). Whether the distinction is equally reliable on real drives with their speed fluctuations has not yet been established.

6.5.7 Exporting as a G64 file

What MS-NIBCOPY512 has read can be saved as a **G64 file** – the usual format for bit-accurate disk images, used for example by VICE, the SD2IEC and the C64 Ultimate. The destination is **another drive** with enough space, typically an **SD2IEC**: a G64 file is about 300 KB in size and does not fit on a 1541 disk.

1. First read the disk: **READ DISK** (or COPY DISK). Without data that has been read, the program reports `NO NIBBLE DATA IN STORAGE`.
2. Select **EXPORT G64 FILE** and press RETURN.
3. `EXPORT TO DRIVE: 09`: select the destination drive 8–11 with **CRSR left/right**, **RETURN** accepts. The program checks whether the device answers (otherwise `DRIVE 10: NOT PRESENT`). The suggestion is the first drive that is not the read drive, afterwards the one last used.
4. `FILE NAME: NIBCOPY.G64`: type the name (up to 12 characters: A–Z, 0–9, space, `-`, `.`, `+`; **DEL** deletes). The extension `.G64` is appended; an existing file of the same name is replaced. **RETURN** starts the export, ← aborts.
5. The track grid shows the progress (`EXPORTING TRACK nn`). At the end, `EXPORT OK: NAME.G64` and the message of the destination drive appear, e.g. `DRIVE 09: 00, OK,00,00`.



Figure 13: Export from MS-RAM finished: file MEINSPIEL.G64 on drive 9 (e.g. an SD2IEC)

- Each track is aligned as when writing to disk and built with complete syncs; its length corresponds to the capacity of a track of this density at 300 rpm (longer tracks keep their length). Tracks 36–42, half tracks, killer tracks and weak bits are retained.
- With **MS-RAM** storage, the read drive supplies the data track by track over the parallel cable; it therefore cannot itself be the destination (`MS-RAM USES THIS DRIVE - CHOOSE ANOTHER`). The motor of the read drive does not run during this. With **REU** storage, the destination may be any other device.
- The duration depends on the destination: a few minutes over the normal serial bus, considerably less with a JiffyDOS-capable destination (e.g. SD2IEC) – the C64 kernal DolphinDos26 selects JiffyDOS automatically.

Message	Meaning
`EXPORT OK: NAME.G64`	file written completely
`DRIVE nn: NOT PRESENT`	no device answers at the selected drive
`MS-RAM USES THIS DRIVE - CHOOSE ANOTHER`	MS-RAM storage: read drive selected as destination
`EXPORT FAILED: …`	aborted with RUN/STOP or transfer error; the message of the destination drive is shown below (e.g. `72,DISK FULL`)

6.5.8 Importing a G64 file

The reverse route: a **G64 file** – e.g. an image from VICE, from nibtools or one exported earlier – is loaded from a drive into memory and then written to a real disk with **WRITE DISK**. The source is typically an **SD2IEC**.

1. Select **IMPORT G64 FILE** and press RETURN.
2. `IMPORT FROM DRIVE: 09`: select the drive 8–11 holding the file with **CRSR left/right**, **RETURN** accepts (a missing device reports `DRIVE 10: NOT PRESENT`).
3. `FILE NAME:`: type the name without `.G64` (up to 12 characters) and press RETURN. The suggestion is the name last used, the first time NIBCOPY.
4. The file is read track by track (`IMPORTING TRACK nn`), the grid shows the progress. At the end, `IMPORT OK: TRACKS 01-35` appears (or up to the last track of the file); END TRACK in the main menu then shows this last track.

5. Insert the target disk in the Multi-Speeder drive, select **WRITE DISK** and write as when copying (with verify, then `ANOTHER COPY?`).



Figure 14: Import finished: track 1–35 from MEINSPIEL.G64 in MS-RAM, ready for WRITE DISK

- Each track of the file is analysed like a track that has been read (length, alignment, sectors); the bytes are taken over unchanged when writing. This also retains short syncs, killer tracks, tracks without sync, half tracks and weak bits.
- The tracks must be in ascending order in the file, as with VICE, nibtools and EXPORT G64 FILE; otherwise `IMPORT FAILED: TRACKS NOT IN ORDER` appears. Files with changing density within a track are taken over with the standard density of the zone.
- With **MS-RAM** storage, each track is transferred over the parallel cable into the RAM of the Multi-Speeder; the read drive therefore cannot be the source at the same time. The motor does not start during this (only once at the first access, when the drive reads the BAM). With **REU** storage, the source may be any drive.

Message	Meaning
`IMPORT OK: TRACKS 01-35`	file loaded, ready for WRITE DISK
`IMPORT FAILED: NOT A VALID G64 FILE`	not a G64 file, file too short or faulty
`IMPORT FAILED: TRACKS NOT IN ORDER`	tracks of the file not in ascending order
`DRIVE 09: 62,FILE NOT FOUND,00,00`	file not found on the drive (message of the drive)

6.5.9 Why slower than nibtools on a PC?

MS-NIBCOPY512 follows the same method as **nibtools**, but takes noticeably longer than nibtools on a PC with an XUM1541 (ZoomFloppy). This is not a fault; it comes mainly from where the computing happens. One full track when reading, measured in the emulator:

Step	Time	Who does the work
Read track	220 ms	drive
Transfer 8 KB to the C64	363 ms	drive and C64, over the parallel cable
Count syncs	126 ms	C64

Step	Time	Who does the work
Determine track start, length and alignment	335 ms	C64
Check sectors	389 ms	C64

- **Computing power:** More than half of the time goes into the analysis. nibtools does the same steps (density, track length, alignment, sectors, invalid GCR) on the PC, where they hardly matter. Here the 8-bit processor of the C64 does the work at about 1 MHz.
- **Drive:** This part makes no big difference. nibtools also reads with the same mechanism and, on a 1541, fetches the raw data over a parallel cable, 8 KB per track.
- **Memory:** The C64 has 64 KB of RAM. The tracks therefore sit in the 8 KB banks of the Multi-Speeder or in the REU and must be transferred to the C64 for analysis.
- **Checking when writing:** Here VERIFY by default reads each track back and compares it, with up to 3 attempts. In nibtools this check is off by default. VERIFY OFF (EDIT PRESET) makes writing faster, but write errors then go unnoticed.
- **Difficult tracks:** Tracks with invalid GCR take several reads on both sides. On the C64 the comparison comes on top (see Weak bits).

6.5.10 Test status

- Also in the emulator: DIRECTORY (including with read tracks in MS-RAM, which are then written unchanged), END TRACK after an import, the file name suggestion with arbitrary RAM contents, and the message for a missing drive (selection, DIRECTORY, SAVE PRESETS, import, program start).
- In the VICE emulator with an emulated Multi-Speeder: reading and writing with verify from MS-RAM and REU, half tracks, track 36–42, TRACK INFO, saving and loading presets – the copy matches the source bit by bit, the files are readable.
- Copy protection (G64 test images): 10 of 11 copies start, one of them not with every copy (see Weak bits). This means the V-MAX protection with very short syncs on track 20 can also be copied. Not yet copyable is a test image whose loader checks places with four and five zero bits (see Four and five zero bits) also on tracks 31–38. These tracks have density 0 and cannot be read more slowly.
- G64 export and import from and to MS-RAM and REU: the exported file matches the source track by track and in the densities; disks written from the imported file pass the verify, the copy starts.
- The red LED is lit during reading and writing and is off afterwards; during import and export with MS-RAM, the motor does not start for every track.
- Tests with real disks on the Multi-Speeder have not yet been carried out.

> **Legal notice:** Copies may only be made of disks to which you hold the rights (e.g. backup copies of your own originals).

7 DolphinDos26UL

DolphinDos26UL (file `C1541_DOLPHINDOS26UL.bin`, 32 KB) is a reduced version of DolphinDos26 for drives **without** Multi-Speeder: parallel cable on VIA1 (as with DolphinDOS 2.0, no 6821 PIA) and 8 KB RAM at $6000–$7FFF. It is intended for the **Commodore 64 Ultimate**, which emulates exactly this cable and RAM, and for real 1541 drives with DolphinDOS 2 hardware. Setup, operation and MS-NIBCOPY512 with REU are described in its own manual `DolphinDos26UL_Handbuch.odt` in the `Builds` folder.

Feature	DolphinDos26 (Multi-Speeder)	DolphinDos26UL
ROM	`1541DolphinDos26.bin` (Bank 8)	`C1541_DOLPHINDOS26UL.bin`, 32 KB
Cable	6821 PIA at $5000 (DolphinDOS 3)	VIA1 port A (DolphinDOS 2)
Drive RAM	512 KB in 64 banks	8 KB at $6000–$7FFF
Parallel, track cache, @/X commands, 40 tracks	yes	yes
CDR/CDW, RAM disk CDM, MS-Addon-API	yes	no (`31,SYNTAX ERROR`)
MS-MEMTEST512	yes	no
MS-FASTCOPY512	MS-RAM or REU	REU only
MS-NIBCOPY512	MS-RAM or REU	REU only
Power-on message	`73, DOLPHINDOS26 1541`	`73, DOLPHINDOS26 C64U`

> The drive ROMs are **not interchangeable**: DolphinDos26 transfers via the PIA of the Multi-Speeder, DolphinDos26UL via VIA1. With the wrong ROM, the parallel transfer hangs.

8 DolphinDos26DD2

DolphinDos26DD2 (file `C1541_DOLPHINDOS26DD2.bin`, 64 KB) is DolphinDos26 for the Multi-Speeder V3 with a different parallel cable: the cable is not connected to the PIA but, as with DolphinDos26UL, to VIA1 of the drive (SpeedDOS cable, port A $1801, handshake CB1/CA2). All DolphinDos26 functions remain, including CDR, CDW, CDM and the MS-Addon-API. The PIA now only switches the RAM banks.

The advantage: on the 1541 and 1541-II a single parallel cable is enough, and it stays usable for programs that use the SpeedDOS cable, for example Burst Nibbler, 15 Second Copy or tools for DolphinDOS 2.

Feature	DolphinDos26	DolphinDos26DD2
ROM (bank 8)	`1541DolphinDos26.bin`	`C1541_DOLPHINDOS26DD2.bin`
Parallel cable	6821 PIA at $5000	VIA1 port A $1801
PIA	parallel port and RAM banks	RAM banks only; port A and CA2 are inputs
Power-on message	`73, DOLPHINDOS26 1541`	`73, DOLPHINDOS26  DD2`
Random RAM at power-on	not initialised	initialised as in DolphinDos26UL; `UJ` keeps settings and RAM disk
MS-FASTCOPY512	MS-RAM or REU	MS-RAM or REU; the display reads `PARALLEL: VIA1`
MS-NIBCOPY512	MS-RAM or REU	MS-RAM or REU, from Build 48

Setting up:

1. In your own 512 KB image, replace bank 8 from offset $70000 with `C1541_DOLPHINDOS26DD2.bin` and burn the EPROM. A ready-made 512 KB image with this bank is not included.
2. Connect the parallel cable to port A of VIA1 (as with the SpeedDOS cable). The C64 kernal `C64DolphinDos26.bin` stays unchanged.
3. Select bank 8 and check with `@`: the message reads `73, DOLPHINDOS26  DD2`.

> **PIA lines:** Every drive reset, including `UJ`, sets port A of the PIA and CA2 to input. If the parallel lines of the PIA are connected 1:1 to the cable on VIA1, the PIA therefore does not drive them.

> Firmware and cable must match: DolphinDos26 needs the cable on the PIA, DolphinDos26DD2 the cable on VIA1. MS-NIBCOPY512 up to Build 47 reports `NO ANSWER FROM DRIVE CODE` with DolphinDos26DD2.

Test status: Checked in the VICE emulator with Multi-Speeder and standard parallel cable: LOAD, SAVE and VERIFY, CDR, CDW and CDM, XQ and XZ from the RAM disk, formatting, reset and PIA state, MS-NIBCOPY512 (bit-exact reading and writing) and reading with MS-FASTCOPY512. DolphinDos26DD2 also runs on real hardware (confirmed by the developer); checked on the C64U with a 1541: 160 loads of 32 KB each over the VIA1 cable without errors, MS-NIBCOPY512 with MS-RAM and REU. Not yet tested are the 1:1 connection of the PIA lines and the third-party programs named above.

8.1 DolphinDos26DD2 and DolphinDos26UL compared

Both DOS ROMs transfer in parallel via the drive's VIA1, but they are meant for different hardware. `C1541_DOLPHINDOS26DD2.bin` supports the Multi-Speeder's MS-Addons and uses VIA1 for the parallel transfer. `C1541_DOLPHINDOS26UL.bin` is specifically compatible with DolphinDOS 2 and needs only the 8 KB track cache and a DolphinDOS 2 / SpeedDOS 2 compatible parallel cable on VIA1. It does not have the MS-Addons such as the internal disk copy and the RAM disk, because they need the 512 KB of the Multi-Speeder V3.

Feature	C1541_DOLPHINDOS26DD2.bin	C1541_DOLPHINDOS26UL.bin
Intended for	1541 with Switchless Multi-Speeder V3 (512 KB)	DolphinDOS 2 hardware (8 KB track cache) and the C64U
Size	64 KB (bank 8 of the EPROM)	32 KB ($8000–$FFFF)
Parallel transfer	VIA1 (SpeedDOS cable)	VIA1 (DolphinDOS 2 / SpeedDOS 2 compatible cable)
Drive memory	512 KB of the Multi-Speeder	only 8 KB track cache at $6000
MS-Addons (internal disk copy CDR/CDW, RAM disk CDM, MS-Addon API)	yes	no
MS-FASTCOPY512 / MS-NIBCOPY512	MS-RAM or REU	REU only

9 Performance data

Measured in the VICE emulator (C64 PAL, 1541 with Multi-Speeder), real time:

Operation	Time
Load 30 KB program – parallel (DolphinDOS)	3.7 s
Load 30 KB program – serial (CBM kernal)	66 s
Load 30 KB program – from the RAM disk	1.2 s
Save 30 KB program – disk / RAM disk	7.3 s / 1.9 s
Read disk (35 tracks) into RAM – CDR1	22.9 s
Write RAM to disk – CDW1 (with verify)	26.3 s
MS-MEMTEST512 – one run (512 KB, 4 patterns)	approx. 2 min
MS-NIBCOPY512 – read and analyse track / write with verify	approx. 1.5 s / 2.5 s

10 Quick reference of the new commands

Command	Effect
`8@RNROM`	switch to DolphinDos26 (e.g. `@I:8@RNROM`)
`CDR1` / `CDR2`	read disk into RAM side 1 / 2 (35/40 tracks automatically)
`CDR1+` / `CDR2+`	read 40 tracks
`CDW1` / `CDW2`	write RAM side 1 / 2 to disk
`CDM1` / `CDM2`	switch on RAM disk from side 1 / 2
`CDM0`	RAM disk off, back to the disk
`M-E $3FF9 … $3FE7`	MS-Addon-API E0–E6 (see 5.5)

Error messages: an unknown C command results in `31,SYNTAX ERROR`; read and write errors come with the DOS error code, e.g. `21,READ ERROR`, `23,READ ERROR`, `25,WRITE ERROR`, `26,WRITE PROTECT ON`.

11 Notes and limitations

- All functions have been tested in the VICE emulator with an emulated Multi-Speeder. The DolphinDos26 and DolphinDos26DD2 firmware, the C64 kernal, also as C64 II ROM, and MS-FASTCOPY512 (also with REU storage) run on real hardware; MS-NIBCOPY512 reads and writes normal disks there with MS-RAM and REU, copy-protected disks have not yet been tested on real hardware. The red LED and D64 import and export of MS-FASTCOPY512 have so far only been tested in the emulator. The case “target disk write-protected” with CDW is still untested.
- The contents of the 512 KB RAM (CDR sides, RAM disk) survive a reset of the drive, but not power-off.
- MS-MEMTEST512 and MS-NIBCOPY512 (MS-RAM storage) overwrite the RAM contents.
- A DolphinDOS kernal is needed for the C64 part (parallel transfer, @ commands), preferably `C64DolphinDos26.bin`. With the original CBM kernal, the commands work serially via `OPEN 1,8,15,…`.
- Programs with their own fast loader or copy protection sometimes run only with the original C64 kernal and original DOS (bank 1, `1@RNROM`); titles with “NTSC” or “USA” in the name only on an NTSC C64.
- Like DolphinDOS, DolphinDos26 uses illegal NMOS 6502 instructions and therefore does not run with a WDC 65C02.

12 Appendix

12.1 Allocation of the RAM banks

Logical bank	DolphinDos26 (CDR/CDW/CDM)	MS-NIBCOPY512 (MS-RAM storage)
0	working RAM of DolphinDOS (variables, track cache, state of CDR/CDM)	–
1–22	side 1, track 1–35	track 1–22 (bank t = track t)
23–42	side 2, track 1–35 (beginning)	track 23–42
43–44	side 2, track 1–35 (end)	half tracks
45–47	side 1, track 36–40	half tracks
48–50	side 2, track 36–40	half tracks
51–62	free	half tracks
63	free	read and write buffer

MS-NIBCOPY512 with MS-RAM storage therefore overwrites the CDR sides and a RAM disk.

12.2 Origins and acknowledgements

Project	Developer	Contribution
DolphinDOS 3.0	Günther Jilg (software) and Jan Bubela (hardware), Dolphin Software 1985–1990	basis of the drive ROM and C64 kernal
JiffyDOS	Mark Fellows; Creative Micro Designs (CMD), today Retro Innovations	JiffyDOS protocol in the C64 kernal DolphinDos26
nibtools	Pete Rittwage, based on MNIB by Markus Brenner	methods and algorithms of MS-NIBCOPY512
Exomizer	Magnus Lind	packer of the programs on `ms-tools.d64`
VICE	the VICE team	emulator in which everything was developed and tested

Special thanks go to Jim Brain of RETRO Innovations for his support with the JiffyDOS implementation of the C64/C128 kernal DolphinDos26.

> **Please check: JiffyDOS licence.** The kernal contains JiffyDOS transfer routines that were taken over instruction for instruction from a JiffyDOS kernal; the JiffyDOS firmware for the drive is not included. Jim Brain describes it as follows: no licence is needed for code that implements the same transfer behaviour but is not an exact copy of the JiffyDOS routines. If the transfer code is identical, then in his assessment a licence would be required on the part of the end user. Every user is therefore asked to check for themselves whether they should purchase an official JiffyDOS licence because of their use. Further information: www.jiffydos.com

Multi-Speeder, DolphinDos25/26, C64 kernal DolphinDos26 and MS-Tools: **Frank Eggen**. The fully commented source code (ca65) is in the folders `dd25` (firmware), `C64DD26` (kernal) and `MS-Tools` (C64 programs).

Topic	Address
Contact	frankeggen@emden.net
Projects on GitHub	https://github.com/fraegg

12.3 Disclaimer

DolphinDos26, the C64 kernal DolphinDos26 and the MS-Tools are a **private hobby project** and are provided without any warranty. Use is **at your own risk**; no liability is accepted for damage to hardware, disks or data or for any other consequences. Formatting, writing and nibble copying change disks irreversibly – make backup copies first. Copyright must be observed when copying disks.

12.4 Support through a donation

Anyone who wishes to support the project can voluntarily send the developer a donation via PayPal:

> **PayPal:** Donation to Frank Eggen

12.5 Copyright and terms of use

Part	Rights holder
Extensions of DolphinDos26, C64 kernal DolphinDos26, MS-Tools, this manual	© 2025–2026 Frank Eggen
DolphinDOS 3.0	© Günther Jilg and Jan Bubela, Dolphin Software
JiffyDOS	© Mark Fellows / Creative Micro Designs; licensing today via Jim Brain, RETRO Innovations
CBM DOS 1541, C64 kernal and BASIC	© Commodore
nibtools	© Pete Rittwage and contributors

The software and this manual may be used and passed on for **private, non-commercial use**, provided that they remain unchanged and this notice is retained. **No sale and no commercial use without consulting the developer** (in particular, sale of ROM files or programmed EPROMs). Please send enquiries to frankeggen@emden.net.